

목차 (Table of Contents)

1. CPU-GPU 스와핑 비효율의 근본 원인
 - 1.1 문제 정의
 - 1.2 실험으로 밝혀진 근본 원인들
 - 1.3 결론: 456배 느린 이유
2. Unified Memory 동작 분석 결과
 - 2.1 메모리 할당 동작
 - 2.2 Memory Stats 분석
 - 2.3 고빈도 모니터링 결과
 - 2.4 cudaMallocManaged 동작
3. 프리페칭 실험 결과
 - 3.1 cudaMemPrefetchAsync 분석
 - 3.2 프리페치와 연산 중첩
 - 3.3 전송 Granularity 분석
 - 3.4 Alpamayo 모듈별 전송 시간 (Pinned Memory)
 - 3.5 메모리 압력에 따른 접근 시간
4. 명시적 텐서 핀닝 실험 결과
 - 4.1 Pinned vs Pageable 대역폭 비교
 - 4.2 비동기 스트림 전송 비교
 - 4.3 레이아웃별 오프로딩 시뮬레이션
 - 4.4 PCIe 효율 (WSL2)
5. WSL2 오버헤드 분석 결과
 - 5.1 전송 지연 시간 (Latency) 분석
 - 5.2 Pinning 오버헤드
 - 5.3 WSL2 환경 요약
6. 최적화 전략 제안
 - 6.1 전략 A: 명시적 Pinned Memory 기반 모듈 스와핑 (강력 추천)
 - 6.2 전략 B: 최적 전송 Granularity 활용
 - 6.3 전략 C: 양자화 + Pinned Memory 조합
 - 6.4 전략 D: 하이브리드 GPU-상주 + 스와핑
7. 적용 가능성 판단 + 예상 효과/한계
 - 7.1 적용 가능성 평가

7.2 핵심 한계

7.3 방향 2 결과와의 비교/수정

7.4 수정된 Alpamayo 전송 시간 추정 (Pinned Memory 기준)

8. 시각화 참조

부록: 실험 데이터 파일

부록: 방향 1, 2와의 연결

방향 1에서의 발견

방향 2에서의 발견 (본 연구에서 수정)

본 연구(방향 3)의 고유 기여

결합 전략

연구 방향 3: CPU-GPU 메모리 스와핑 비효율 해결 분석 보고서

실험일: 2026-02-25 환경: NVIDIA GeForce RTX 3080 Ti (12 GB VRAM), 15 GB System RAM, 4 GB Swap 모델: nvidia/Alpamayo-R1-10B (11.08B params, BF16 = 22.16 GB) PyTorch 2.8.0+cu128, CUDA 12.8, WSL2 (Kernel 6.6.87.1-microsoft-standard-WSL2) PCIe: Gen3 x16 (이론 최대 15.75 GB/s)

1. CPU-GPU 스와핑 비효율의 근본 원인

1.1 문제 정의

FP16 Alpamayo baseline 추론 시 273.79초가 소요되며, 이는 이론적 스왑 시간(~0.6초) 대비 약 456배 느리다. Peak VRAM 21.52GB vs 물리 12GB로 약 9.5GB가 Unified Memory에 의해 자동 스와핑된다.

1.2 실험으로 밝혀진 근본 원인들

본 연구에서 4가지 핵심 원인을 실험적으로 확인했다:

원인 1: CUDA Unified Memory의 비효율적 페이지 관리 (WSL2)

- `cudaMemPrefetchAsync(device=-1, CPU)` 가 WSL2에서 미지원 (“invalid device ordinal”)
- GPU에서 할당된 Unified Memory는 `cudaMemset` 접근 시 ~5.7 GB/s의 유효 대역폭 (pinned 전송의 7.8 GB/s 대비 27% 느림)
- Cold/Warm 접근 시간 차이가 거의 없음 (1.0x~1.09x) → WSL2에서 Unified Memory 페이지가 GPU에 영구 상주하거나, prefetch hint가 무시되는 것으로 추정

원인 2: Pageable Memory D2H 전송의 비정상적 저속

전송 방향	Pinned	Pageable	Pageable 감속 비율
H2D (CPU→GPU)	8.5 GB/s	7.0 GB/s	1.2x
D2H (GPU→CPU)	11.8 GB/s	2.6 GB/s	4.5x

- WSL2에서 Pageable D2H가 Pinned 대비 **4.5배 느림** (매우 비정상적)
- 방향 2에서 측정한 D2H 2.48 GB/s는 pageable memory 사용으로 인한 것이었음
- **Pinned memory 사용 시 D2H는 실제로 H2D보다 빠르다** (11.8 vs 8.5 GB/s)

원인 3: 메모리 오버커밋 시 할당/접근 시간 급증

할당량	할당 시간	접근 시간 (sum())
8 GB (VRAM 내)	0.22s	57.7 ms
10 GB (VRAM 내)	0.28s	11.5 ms
12 GB (VRAM 경계)	0.40s	13.7 ms
14 GB (초과)	1.37s	357.4 ms
16 GB (초과)	1.83s	589.9 ms
18+ GB	OOM	-

- 12GB를 초과하면 할당 시간이 **3.4배**, 접근 시간이 **26배** 급증
- 순차 1GB 청크 할당에서: VRAM 내(~30ms/청크) → 초과 시(~250ms/청크, **8.3배**)

원인 4: 양방향 전송 경합

전송 모드	시간 (256MB)	대역폭
H2D 단독	34.5 ms	7.78 GB/s
D2H 단독	22.6 ms	11.86 GB/s
양방향 동시 (총 시간)	46.8 ms	-
순차 (H2D + D2H)	57.1 ms	-

- 양방향 동시 전송은 순차 대비 **1.22x** 중첩 효과
- 그러나 D2H가 양방향 시 **2.05x 감속** → Unified Memory의 동시 eviction + fetch 시 심각한 경합

1.3 결론: 456배 느린 이유

Alpamayo의 273.79초 추론이 이론 대비 456배 느린 것은 다음 요인의 **복합적 결과**:

1. **페이지 폴트 기반 스와핑**: Unified Memory가 4KB~2MB 단위로 on-demand 페이지 전송 → 소형 전송의 대역폭 효율 저하 (1MB 이하: ~2.5 GB/s)
2. **WSL2 Pageable D2H 비정상**: eviction 시 D2H 전송이 2.6 GB/s로 pinned 대비 4.5x 느림
3. **오버커밋 접근 시간 급증**: 12GB 초과 분(~9.5GB)에 대한 접근 시 지연 26x 증가
4. **양방향 경합**: 새 페이지 fetch와 기존 페이지 eviction이 동시에 발생, D2H 2x 감속
5. **예측 불가능한 접근 패턴**: Transformer의 attention 연산은 비순차적 메모리 접근 → 페이지 폴트 빈도 극대화
6. **Prefetch 미지원**: WSL2에서 cudaMemPrefetchAsync(CPU) 미동작 → 선제적 스와핑 불가

2. Unified Memory 동작 분석 결과

2.1 메모리 할당 동작

PyTorch(CUDA 12.8)는 12GB 물리 VRAM을 초과하여 최대 약 24GB까지 할당 가능하다. 이는 CUDA Unified Memory의 oversubscription 기능에 의한 것이다.

순차 1GB 청크 할당 실험:
Chunk 0~10 (0~11GB, VRAM 내): ~30ms/chunk
Chunk 11~14 (12~15GB, 초과): ~250ms/chunk (8.3x 느림)
총 15GB 할당 가능 (12GB 물리 + ~3GB 스왑)

2.2 Memory Stats 분석

- **num_alloc_retries**: 1 (전체 VRAM 소진 후 재시도)
- **num_ooms**: 1 (24GB+ 할당 시도 시)
- **peak 할당**: 21.48 GB (물리 12GB의 1.79배)
- **oversize_allocations**: 0 (모든 할당이 caching allocator 내)

2.3 고빈도 모니터링 결과

10ms 간격 모니터링에서 관찰된 패턴: - 할당/해제 사이클: 평균 alloc 12ms, compute 33ms, free 16ms - VRAM 충전 시: 16~17ms/512MB 청크로 안정적 - VRAM 가득 찬 상태에서 matmul: 116.7ms (추가 임시 메모리 필요 시 지연)

2.4 cudaMallocManaged 동작

- `cudaMallocManaged` 자체는 WSL2에서 정상 동작
- 할당 시간이 크기에 비례 (100MB: 72ms, 4GB: 2130ms)
- `cudaMemPrefetchAsync(GPU)` : 즉시 반환 (0.001~0.087ms) → 비동기로 배경 전송
- `cudaMemPrefetchAsync(CPU, device=-1)` : **WSL2에서 미지원** (error code 반환)

3. 프리페칭 실험 결과

3.1 cudaMemPrefetchAsync 분석

핵심 발견: WSL2에서 `cudaMemPrefetchAsync(CPU)`가 미지원

이는 Unified Memory의 핵심 최적화 기법인 “선제적 eviction”이 WSL2에서 불가능함을 의미한다. GPU에서 CPU로의 프리페치가 불가능하므로, CUDA 런타임이 페이지 폴트 시에만 수동적으로 eviction을 수행한다.

3.2 프리페치와 연산 중첩

CUDA 스트림을 사용한 전송-연산 중첩 효과:

전송 크기	순차	중첩	시간 절감
128 MB	23.8 ms	17.0 ms	28.6%
256 MB	42.6 ms	34.6 ms	18.7%
512 MB	76.8 ms	68.4 ms	10.8%
1024 MB	145.2 ms	129.5 ms	10.8%

- 전송이 연산보다 지배적이므로, 중첩 효과는 연산 시간에 비례하여 제한적
- 가장 큰 효과는 작은 전송(128MB)에서 28.6% 절감

3.3 전송 Granularity 분석

핵심 발견: 작은 청크(2~8MB)가 단일 대형 전송보다 빠르다

청크 크기	청크 수	총 시간 (512MB)	유효 대역폭
0.5 MB	1024	102.5 ms	5.24 GB/s
1 MB	512	101.3 ms	5.30 GB/s
2 MB	256	44.1 ms	12.16 GB/s
4 MB	128	43.4 ms	12.37 GB/s
8 MB	64	43.2 ms	12.41 GB/s
16 MB	32	44.5 ms	12.08 GB/s
64 MB	8	46.7 ms	11.50 GB/s
256 MB	2	56.6 ms	9.49 GB/s
512 MB	1	69.1 ms	7.77 GB/s

- **최적 청크 크기: 2~8 MB** (12.2~12.4 GB/s, 단일 전송의 **1.6배**)
- 이유: 다수의 비동기 DMA 요청이 파이프라인화되어 PCIe 대역폭을 더 효율적으로 활용
- 0.5~1MB는 per-transfer 오버헤드(~0.36ms)가 지배적이어서 비효율적

이 발견은 Unified Memory의 페이지 단위 전송(4KB~2MB)이 이론적으로는 최적 범위에 있으나, 페이지 폴트 기반의 비동기화 부재로 효율이 저하됨을 시사한다.

3.4 Alpamayo 모듈별 전송 시간 (Pinned Memory)

모듈	크기	H2D (ms)	H2D BW	D2H (ms)	D2H BW
VLM Layer (single)	386 MB	52.3	7.74 GB/s	34.1*	4.35* GB/s
VLM Embedding	280 MB	38.0	7.72 GB/s	25.2	11.65 GB/s
Vision Encoder	1150 MB	138.8	8.69 GB/s	101.4*	3.55* GB/s
VLM LM Head	1280 MB	151.0	8.89 GB/s	114.8	11.69 GB/s
Expert (1/4 scale)	1140 MB	137.1	8.72 GB/s	101.1	11.83 GB/s

(*) 첫 번째 trial에서 cold start 영향이 포함된 평균. 안정 시 D2H는 11.6~11.9 GB/s.

3.5 메모리 압력에 따른 접근 시간

배경 압력	총 할당	VRAM 초과	1GB 할당 시간	접근 시간
2 GB	3.0 GB	NO	29.8 ms	1.2 ms
6 GB	7.0 GB	NO	30.4 ms	1.2 ms
10 GB	11.0 GB	NO	32.3 ms	1.2 ms
11 GB	12.0 GB	YES	54.9 ms	17.4 ms

- 12GB 경계를 넘는 순간 할당 시간 1.7x, 접근 시간 **14.3x** 급증
- 이 “cliff” 효과가 Alpamayo 추론의 핵심 병목

4. 명시적 텐서 핀닝 실험 결과

4.1 Pinned vs Pageable 대역폭 비교

크기	Pinned H2D	Pageable H2D	Pinned D2H	Pageable D2H	D2H 감속
64 MB	8.52 GB/s	6.84 GB/s	11.81 GB/s	2.35 GB/s	5.02x
128 MB	8.84 GB/s	7.11 GB/s	11.90 GB/s	2.55 GB/s	4.67x
256 MB	8.16 GB/s	7.18 GB/s	11.93 GB/s	2.72 GB/s	4.38x
512 MB	8.35 GB/s	7.20 GB/s	11.85 GB/s	2.56 GB/s	4.63x
1024 MB	9.42 GB/s	7.96 GB/s	11.92 GB/s	2.63 GB/s	4.53x

핵심 발견 1: Pinned D2H(11.8 GB/s)가 Pinned H2D(8.5 GB/s)보다 **1.4배 빠르다**

이는 방향 2에서 관찰한 “D2H가 H2D의 1/3” 결과를 뒤집는다. 방향 2는 pageable memory를 사용했기 때문이다. Pinned memory 사용 시 H2D/D2H 대역폭 관계가 역전된다.

핵심 발견 2: WSL2에서 Pageable D2H가 비정상적으로 느림 (2.6 GB/s, pinned의 1/4.5)

WSL2의 가상화 레이어가 pageable D2H에서 추가적인 복사 또는 페이지 관리를 수행하는 것으로 추정된다.

4.2 비동기 스트림 전송 비교

5개 VLM 레이어(200MB/each) 시뮬레이션:

전략	총 시간	스피드업
Blocking 순차	200.4 ms	1.00x
Non-blocking	147.3 ms	1.36x
Double-buffered Pipeline	129.4 ms	1.55x

4.3 레이어별 오프로딩 시뮬레이션

6개 Transformer 레이어(102MB/each) 시뮬레이션:

전략	총 시간	스피드업	평균 H2D	평균 연산
동기식 순차	1777.4 ms	1.00x	275.0 ms	23.0 ms
비동기 파이프라인	1545.3 ms	1.15x	0.05 ms (wait)	216.4 ms

비동기 파이프라인에서 다음 레이어 전송이 현재 레이어 연산과 중첩되어 wait 시간이 0.05ms로 줄어든다. 다만 전체 스피드업은 1.15x로 제한적인데, 이는 전송 시간 > 연산 시간이기 때문이다.

4.4 PCIe 효율 (WSL2)

방향	Pinned BW	이론 BW	효율	WSL2 오버헤드
H2D	9.34 GB/s	15.75 GB/s	59.3%	40.7%
D2H	11.80 GB/s	15.75 GB/s	74.9%	25.1%

- H2D에서 WSL2 오버헤드가 40.7%로 상당함
- 네이티브 Linux에서는 ~12 GB/s (H2D) 이상 달성 가능할 것으로 추정

5. WSL2 오버헤드 분석 결과

5.1 전송 지연 시간 (Latency) 분석

전송 크기	평균 시간	최소 시간	유효 대역폭
4 B	0.408 ms	0.360 ms	~0
1 KB	0.454 ms	0.352 ms	~0
64 KB	0.423 ms	0.318 ms	0.16 GB/s
1 MB	0.484 ms	0.410 ms	2.17 GB/s
16 MB	2.479 ms	2.417 ms	6.77 GB/s
64 MB	9.092 ms	8.979 ms	7.38 GB/s

- 고정 오버헤드(latency): ~0.36 ms per transfer
- 이는 PCIe 3.0의 이론적 TLP 오버헤드(1~3 us)의 **120~360배**
- WSL2 가상화 레이어(Hyper-V → GPU-PV 드라이버)가 이 지연의 주원인

5.2 Pinning 오버헤드

크기	할당 시간	핀닝 시간	핀닝 오버헤드
1 MB	12.5 ms	0.23 ms	1.8%
10 MB	125.4 ms	1.0 ms	0.8%
100 MB	1258.2 ms	8.5 ms	0.7%
500 MB	6451.6 ms	49.4 ms	0.8%

- 핀닝 자체의 오버헤드는 매우 작음 (할당의 <1%)
- 다만, 할당 자체가 상당히 느림 (500MB 할당에 6.5초) → WSL2 특유의 메모리 관리 오버헤드

5.3 WSL2 환경 요약

```
Kernel: 6.6.87.1-microsoft-standard-WSL2
PCIe: Gen3, 16x (current=3, max=3, width=16, max=16)
GPU: RTX 3080 Ti, 12.88 GB VRAM, 80 SMs
```

WSL2 고유의 성능 저하 요인: 1. **GPU-PV (GPU Paravirtualization)**: Windows Host의 GPU 드라이버를 경유 2. **Hyper-V 메모리 관리**: 가상 주소 변환 추가 계층 3. **cudaMemPrefetchAsync(CPU) 미지원**: Unified Memory 최적화 기법 사용 불가 4. **Pageable D2H 비정상**: 4.5x 감속 (pinned 대비) 5. **고정 전송 오버헤드**: 0.36ms per transfer (네이티브 대비 ~100x)

6. 최적화 전략 제안

6.1 전략 A: 명시적 Pinned Memory 기반 모듈 스와핑 (강력 추천)

Unified Memory 자동 스와핑 대신 명시적으로 관리하는 방법:

```
# 핵심 아이디어
for layer in model.layers:
    # 1. Pinned memory에서 GPU로 비동기 전송
    layer_gpu = layer.to("cuda", non_blocking=True)

    # 2. 다음 레이어 프리페치 시작 (별도 스트림)
    with torch.cuda.stream(transfer_stream):
        next_layer_gpu = next_layer.to("cuda", non_blocking=True)

    # 3. 현재 레이어 연산
    output = layer_gpu(input)

    # 4. 현재 레이어 해제 (GPU 메모리 반환)
    del layer_gpu
```

예상 효과: - Pinned H2D: 8.5~9.4 GB/s (pageable의 1.2x) - Pinned D2H: 11.8 GB/s (pageable의 4.5x) - VLM 단일 레이어(386MB) 전송: ~52ms (H2D), ~34ms (D2H) - 파이프라인 스피드업: 1.15~1.55x

6.2 전략 B: 최적 전송 Granularity 활용

실험에서 발견한 2~8MB 최적 청크 크기를 활용:

```
# 레이어 파라미터를 2~8MB 청크로 분할하여 비동기 전송
for param_chunk in layer.split_params(chunk_size=4*1024*1024): # 4MB
    gpu_chunk = param_chunk.to("cuda", non_blocking=True)
```

예상 효과: - 유효 대역폭: 12.4 GB/s (단일 전송의 1.6x) - VLM 단일 레이어 전송: ~33ms (7.77 GB/s 대비 37% 단축)

6.3 전략 C: 양자화 + Pinned Memory 조합

INT4 양자화로 모델 크기를 줄이면 전송 오버헤드도 비례하여 감소:

정밀도	모델 크기	VLM Layer 전송 (H2D)	전체 36 Layer
FP16	22.16 GB	52.3 ms	1,883 ms
INT8	11.08 GB	~26 ms	~940 ms
INT4	5.54 GB	~13 ms	~470 ms

6.4 전략 D: 하이브리드 GPU-상주 + 스와핑

Expert(4.56GB)와 VLM의 일부 레이어(6~7GB)를 GPU에 상주시키고, 나머지만 스와핑:

- GPU 상주: Expert(4.56GB) + 가장 빈번히 접근되는 레이어
- 스와핑: 나머지 VLM 레이어
- 피크 VRAM: ~11GB (12GB 이내)
- 스와핑 대상 크기: ~11GB → 전송 시간 대폭 감소

7. 적용 가능성 판단 + 예상 효과/한계

7.1 적용 가능성 평가

전략	구현 난이도	예상 추론 시간	VRAM 요구	실현성
A: Pinned Memory 스와핑	높음	~150-200s	~6.5 GB	가능
B: 최적 Granularity	중간	~130-180s	~6.5 GB	가능
C: INT4 + Pinned	중간	~80-120s	~5.5 GB	가장 현실적
D: 하이브리드 상주	높음	~100-150s	~11 GB	가능
Baseline (Unified Memory)	N/A	273.79s	21.52 GB	현재

7.2 핵심 한계

1. **시스템 RAM 제약**: 15GB RAM으로는 22GB 모델을 모두 pinned memory에 올릴 수 없음
→ memory-mapped safetensors + 부분 핀닝 필요
2. **Autoregressive 디코딩**: 매 토큰마다 36 레이어 순회 → 토큰당 $36 \times 52\text{ms} = 1,872\text{ms}$ (동기식), 파이프라인으로 $\sim 1,620\text{ms}$
3. **WSL2 고유 문제**: 네이티브 Linux 대비 H2D 40.7% 오버헤드, cudaMemPrefetchAsync(CPU) 미지원
4. **구현 복잡도**: HuggingFace `generate()` 내부 수정 필요

7.3 방향 2 결과와의 비교/수정

본 연구에서 방향 2의 측정 결과를 중요하게 수정해야 한다:

메트릭	방향 2 결과	본 연구 결과 (수정)	원인
H2D 대역폭	7.77 GB/s	8.5~9.4 GB/s (pinned)	Pinned memory 사용
D2H 대역폭	2.48 GB/s	11.8 GB/s (pinned)	Pageable D2H 비정상인 원인
D2H/H2D 비율	1:3 (H2D 우위)	1.4:1 (D2H 우위)	Pinned memory에서 역전
최적 전송 단위	미측정	2~8 MB	Granularity 실험
PCIe 효율	$\sim 50\%$ (H2D)	59.3% (H2D), 74.9% (D2H)	Pinned memory 정밀 측정

이 수정된 결과는 명시적 스와핑의 실용성을 크게 높인다. 특히 D2H(eviction) 대역폭이 기존 측정의 4.8배인 11.8 GB/s라는 것은 레이어 오프로딩의 오버헤드가 당초 예상보다 크게 줄어듦을 의미한다.

7.4 수정된 Alpamayo 전송 시간 추정 (Pinned Memory 기준)

모듈	크기	H2D (추정)	D2H (추정)
Vision Encoder	1.15 GB	131 ms	97 ms
VLM Layer (x36)	0.386 GB/ea	44 ms/ea	33 ms/ea
VLM Embedding	0.28 GB	32 ms	24 ms
VLM LM Head	1.28 GB	145 ms	108 ms
Expert	4.56 GB	518 ms	386 ms

VLM 36 Layer 총 전송 (동기): $36 \times (44 + 33) = \mathbf{2,772 \text{ ms}}$ VLM 36 Layer 총 전송 (파이프라인 1.15x): $\mathbf{2,411 \text{ ms}}$

8. 시각화 참조

그림	파일명	설명
1	<code>figures/01_unified_memory_timeline.png</code>	Unified Memory 고빈도 모니터링 시계열
2	<code>figures/02_bandwidth_comparison.png</code>	Pinned vs Pageable 대역폭 비교 (H2D/D2H)
3	<code>figures/03_granularity_bandwidth.png</code>	전송 단위(Granularity) vs 유효 대역폭
4	<code>figures/04_wsl2_overhead.png</code>	WSL2 오버헤드 종합 분석 (4 패널)
5	<code>figures/05_strategy_comparison.png</code>	스왑 전략 비교 종합 (4 패널)
6	<code>figures/06_transfer_scaling.png</code>	전송 크기별 대역폭 스케일링 + H2D/D2H 비대칭

부록: 실험 데이터 파일

파일	설명
<code>unified_memory_results.json</code>	Part 1 Unified Memory 분석 결과
<code>unified_memory_timeline.csv</code>	Part 1 고빈도 모니터링 시계열
<code>prefetch_results.json</code>	Part 2 프리페칭 실험 결과
<code>pinned_transfer_results.json</code>	Part 3 명시적 텐서 핀닝 실험 결과
<code>pinned_transfer_vram.csv</code>	Part 3 VRAM 모니터링 시계열
<code>wsl2_overhead_results.json</code>	Part 4 WSL2 오버헤드 분석 결과
<code>analyze_unified_memory.py</code>	Part 1 실험 스크립트
<code>test_prefetch.py</code>	Part 2 실험 스크립트
<code>test_pinned_transfer.py</code>	Part 3 실험 스크립트
<code>analyze_wsl2_overhead.py</code>	Part 4 실험 스크립트
<code>create_figures.py</code>	Part 5 시각화 스크립트

부록: 방향 1, 2와의 연결

방향 1에서의 발견

- `device_map="auto"` 는 커스텀 토큰 처리(`fuse_traj_tokens`)와 비호환
- VLM (16.44GB)이 단독으로도 12GB VRAM 초과
- Expert는 VLM의 KV Cache를 직접 참조

방향 2에서의 발견 (본 연구에서 수정)

- Vision Encoder와 Expert는 각각 독립적으로 GPU에 탑재/실행 가능 (실측 확인)
- ~~D2H 대역폭 2.48 GB/s~~ → **Pinned D2H: 11.8 GB/s** (Pageable의 비정상인 원인)
- ~~H2D/D2H 비율 3:1~~ → **Pinned: D2H가 H2D의 1.4배 빠름**

본 연구(방향 3)의 고유 기여

1. **Unified Memory 비효율의 근본 원인 규명**: 페이지 폴트 기반 + WSL2 제약 + Pageable D2H 비정상
2. **cudaMemPrefetchAsync(CPU) WSL2 미지원 확인**: 선제적 eviction 불가
3. **최적 전송 Granularity 발견**: 2~8MB 청크가 단일 전송 대비 1.6x 빠름
4. **Pinned Memory D2H 대역폭 재측정**: 11.8 GB/s (방향 2의 2.48 GB/s에서 4.8x 상향 수정)
5. **양방향 전송 경합 정량화**: 1.22x 중첩, D2H 2.05x 감소
6. **WSL2 고정 오버헤드**: per-transfer 0.36ms (PCIe 이론의 120~360배)

결합 전략

최적의 스와핑 전략은: 1. **Pinned Memory 사용** (Pageable 절대 금지, 특히 D2H) 2. **2~8MB 청크 전송** (단일 대형 전송 대비 1.6x 대역폭) 3. **CUDA 스트림 파이프라인** (1.15~1.55x 스피드업) 4. **INT4 양자화** (전송 데이터량 1/4로 감소) 5. **가능하면 네이티브 Linux** (WSL2 오버헤드 40.7% 제거)